

LAB 4: Motion Planning Using ROS pt.2

By: Vy Tang

ID: 77659006

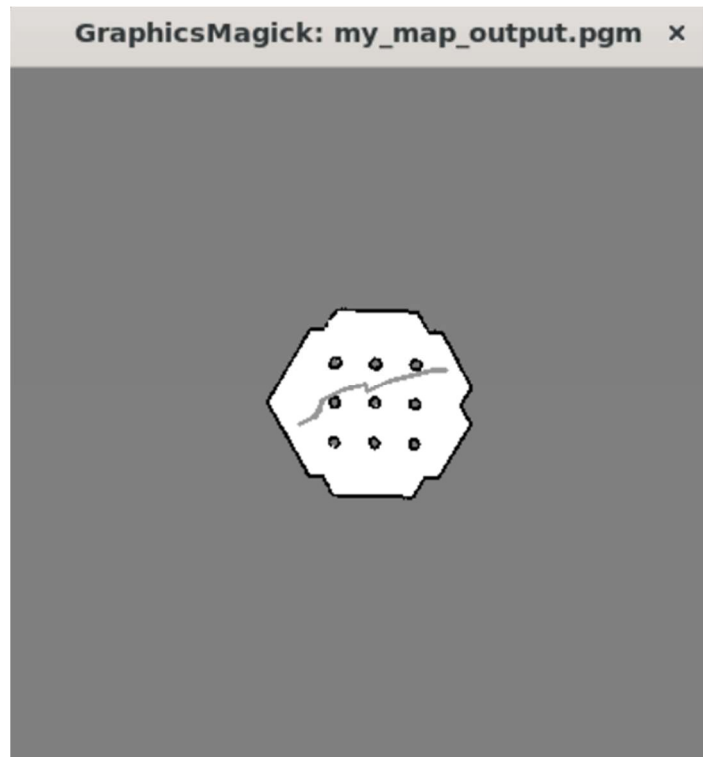
5/5/25

Objective

Test cases

```
Enter your choice (1/2/q): 1  
  
Enter Start and Goal Coordinates (in meters):  
Start x: -2  
Start y: -0.5  
Goal x: 2  
Goal y: 1  
  
Sent start and goal to /start_goal. Waiting for trajectory...  
  
--- Received Trajectory ---
```

```
Point 13: (0.20, 0.55)  
Point 14: (0.40, 0.65)  
Point 15: (0.60, 0.70)  
Point 16: (0.80, 0.75)  
Point 17: (1.00, 0.80)  
Point 18: (1.20, 0.85)  
Point 19: (1.40, 0.90)  
Point 20: (1.60, 0.95)  
Point 21: (1.85, 0.95)  
Point 22: (1.95, 0.95)
```



Test Case 1: The recommended starting test case where the goal is (2,1) and starting point is at (-2,-0.5)

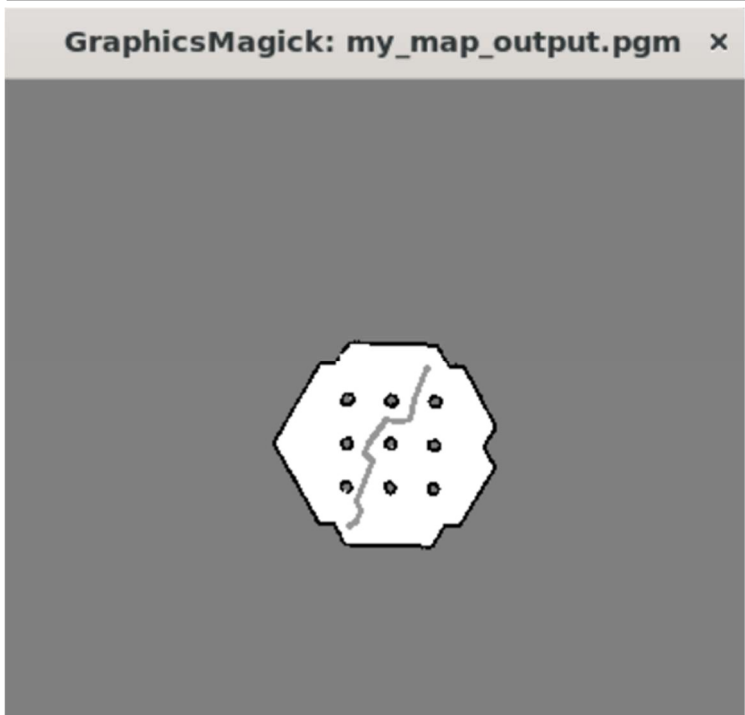
```

q. Quit
Enter your choice (1/2/q): 1

Enter Start and Goal Coordinates (in meters):
Start x: -1
Start y: -2
Goal x: 1
Goal y: 2

Sent start and goal to /start_goal. Waiting for trajectory...

```



```

Point 12: (-0.10, 0.65)
Point 13: (0.10, 0.60)
Point 14: (0.30, 0.60)
Point 15: (0.50, 0.65)
Point 16: (0.55, 0.90)
Point 17: (0.60, 1.15)
Point 18: (0.70, 1.40)
Point 19: (0.80, 1.65)
Point 20: (0.90, 1.90)
Point 21: (0.95, 1.95)

```

Test Case 2: User inputted starting point at (-1, -2) to goal point (1,2)

```

Enter your choice (1/2/q): 2

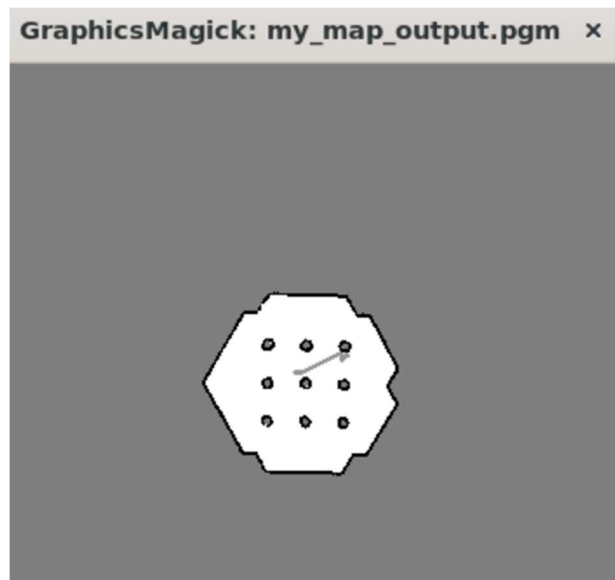
Random Start: (-0.26, 0.40)
Random Goal: (3.42, 1.59)

```

```

-- Received Trajectory ---
Point 0: (-0.30, 0.35)
Point 1: (-0.10, 0.35)
Point 2: (0.10, 0.45)
Point 3: (0.30, 0.55)
Point 4: (0.50, 0.65)
Point 5: (0.70, 0.75)
Point 6: (0.90, 0.85)
Point 7: (1.00, 0.90)
Point 8: (1.05, 0.70)
Point 9: (1.05, 0.75)
Point 10: (1.20, 0.85)

```

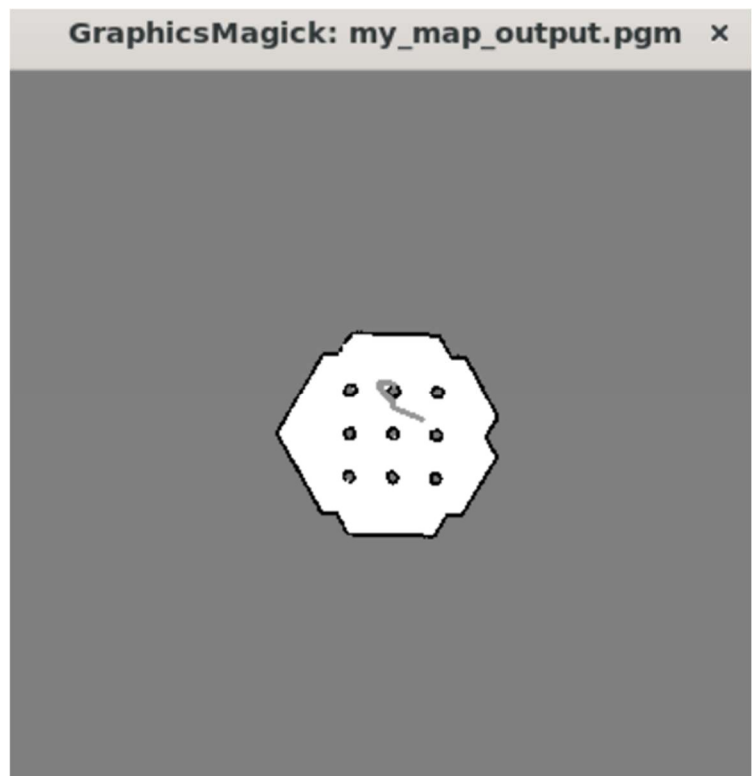


Test Case 3: Randomized start and goal points

```
Enter your choice (1/2/q): 2
```

```
Random Start: (0.76, 0.44)  
Random Goal: (-8.16, 2.88)
```

```
Point 0: (0.75, 0.40)  
Point 1: (0.50, 0.50)  
Point 2: (0.25, 0.60)  
Point 3: (0.00, 0.70)  
Point 4: (0.00, 0.85)  
Point 5: (-0.20, 1.05)  
Point 6: (-0.35, 1.25)  
Point 7: (-0.35, 1.35)  
Point 8: (-0.10, 1.35)  
Point 9: (0.05, 1.30)
```



Test Case 4: Randomized start and goal points

In total there are four test cases. The first test case is the goal and trajectory that was taken in the lab manual and the second test case is a user input test case. The latter two test cases are randomized.

[Code Breakdown](#)

[rrt_node.py](#)

```

1  #!/usr/bin/env python3
2  import rospy
3  import numpy as np
4  import cv2
5  import os
6  from nav_msgs.msg import OccupancyGrid
7  from std_msgs.msg import Float64MultiArray
8  from visualization_msgs.msg import Marker
9  from geometry_msgs.msg import Point
10 from rrt import rrt_pathfinder
11
12 class RRTNode:
13     def __init__(self):
14         rospy.init_node('rrt_node', anonymous=True)
15
16         # Map parameters
17         self.map_img = None
18         self.resolution = None
19         self.origin = None
20
21         # Publishers and Subscribers
22         self.trajectory_pub = rospy.Publisher('/trajectory', Float64MultiArray, queue_size=10)
23         rospy.Subscriber('/map', OccupancyGrid, self.map_callback)
24         rospy.Subscriber('/start_goal', Float64MultiArray, self.start_goal_callback)
25
26         rospy.loginfo("RRT node initialized. Waiting for /map and /start_goal...")
27
28     def occupancygrid_to_image(self, msg):
29         """Convert OccupancyGrid message to image format"""
30         self.resolution = msg.info.resolution
31         self.origin = [msg.info.origin.position.x, msg.info.origin.position.y]
32         width, height = msg.info.width, msg.info.height
33
34         data = np.array(msg.data).reshape((height, width))
35         img = np.full_like(data, 255, dtype=np.uint8)
36         img[data == -1] = 127 # unknown
37         img[data == 100] = 0 # occupied
38         self.map_img = np.flipud(img) # Flip to match image coordinates
39
40     def map_callback(self, msg):
41         """Handle incoming map data"""
42         self.occupancygrid_to_image(msg)
43         rospy.loginfo_once("Map received and converted.")
44
45     def start_goal_callback(self, msg):
46         """Handle start and goal positions and compute path"""
47         if self.map_img is None:
48             rospy.logwarn("Map not yet received. Ignoring goal.")

```

```

49         return
50
51     try:
52         # Extract start and goal positions
53         x_start, y_start, x_goal, y_goal = msg.data
54         start, goal = [x_start, y_start], [x_goal, y_goal]
55         rospy.loginfo(f"Received start: {start}, goal: {goal}")
56
57         # Compute path using RRT
58         _, _, path_px, path_world = rrt_pathfinder(
59             start, goal, self.map_img, self.resolution, self.origin
60         )
61
62         # Publish trajectory
63         self.publish_trajectory(path_world)
64
65         # Visualize and save path
66         self.visualize_path(path_px)
67
68     except Exception as e:
69         rospy.logerr(f"RRT pathfinding failed: {e}")
70
71     def publish_trajectory(self, path_world):
72         """Publish the computed path as trajectory"""
73         flat_path = [coord for point in path_world for coord in point]
74         traj_msg = Float64MultiArray(data=flat_path)
75         self.trajectory_pub.publish(traj_msg)
76         rospy.loginfo("Trajectory published.")
77
78     def visualize_path(self, path_px):
79         """Draw path on map and save to file"""
80         drawn_img = self.map_img.copy()
81
82         # Draw path lines
83         for i in range(1, len(path_px)):
84             pt1 = tuple(map(int, path_px[i - 1]))
85             pt2 = tuple(map(int, path_px[i]))
86             cv2.line(drawn_img, pt1, pt2, color=150, thickness=2)
87
88         # Save image
89         output_path = os.path.join(
90             os.path.dirname(os.path.abspath(__file__)),
91             "..", "..", "..", "my_map_output.pgm"
92         )
93         cv2.imwrite(output_path, drawn_img)

```

```

94         rospy.loginfo(f"Path image saved at {output_path}")
95
96     def run(self):
97         rospy.spin()
98
99 if __name__ == '__main__':
100     try:
101         node = RRTNode()
102         node.run()
103     except rospy.ROSInterruptException:
104         pass

```

In this file, I started by initializing the `rrtnode` class that initializes the `rrt` node. The constructor in the `init` sets up map parameters i.e. `map_img`, `resolution`, and `origin` as well as creating publishers/subscribers for trajectory data, map data, and start/goal positions.

Next is about the map conversion and the function `occupancygrid_to_image` essentially converts ROS occupancy grid messages into a numpy image array where unknown areas are defined as gray at '127', the occupied areas as black at '0', and free space as white at '255'. There is also the map callback function to process new maps when it arrives via the `/map` topic using the `occupancygrid_to_image` function.

For the path planning, the `start_goal_callback` function will check for map availability and handle incoming start/goal positions. If there's map availability, then it will call the RRT pathfinder. After receiving the path from the RRT, it will publish the coordinates as a trajectory and visualize the path on the map.

Now, after receiving the path, the program will publish the trajectory by converting the path into a ROS `Float64MultiArray` message and publish it to the `/trajectory` topic. The `visualize_path` function will then be responsible for actually drawing out the mapped path indicated by the gray lines shown in the test cases above. The last block is to execute the program.

Motion_planner.py

```
1  #!/usr/bin/env python3
2
3  import rospy
4  from std_msgs.msg import Float64MultiArray
5  import random
6  from typing import List, Optional
7
8  class MotionPlanner:
9      def __init__(self):
10         self.trajectory_received = False
11         rospy.init_node('motion_planner', anonymous=True)
12
13         # Publishers and Subscribers
14         self.start_goal_pub = rospy.Publisher('/start_goal', Float64MultiArray, queue_size=10)
15         rospy.Subscriber('/trajectory', Float64MultiArray, self.trajectory_callback)
16
17         # Configuration
18         self.workspace_bounds = {
19             'xmin': -10, 'xmax': 9,
20             'ymin': -10, 'ymax': 9
21         }
22
23         rospy.sleep(1) # Allow time for setup
24
25     def trajectory_callback(self, msg: Float64MultiArray) -> None:
26         """Callback to process and display received trajectories"""
27         self.trajectory_received = True
28         path = msg.data
29         print("\n--- Received Trajectory ---")
30         for i in range(0, len(path), 2):
31             print(f"  Point {i//2}: ({path[i]:.2f}, {path[i+1]:.2f})")
32
33     def get_user_coordinates(self) -> List[float]:
34         """Get start and goal coordinates from user input"""
35         print("\nEnter Start and Goal Coordinates (in meters):")
36         while True:
37             try:
38                 x_start = float(input("Start x: "))
39                 y_start = float(input("Start y: "))
40                 x_goal = float(input("Goal x: "))
41                 y_goal = float(input("Goal y: "))
42                 return [x_start, y_start, x_goal, y_goal]
43             except ValueError:
44                 print("Invalid input. Please enter numeric values.")
```



```

def generate_random_coordinates(self) -> List[float]:
    """Generate random start and goal coordinates within workspace bounds"""
    coords = [
        random.uniform(self.workspace_bounds['xmin'], self.workspace_bounds['xmax']),
        random.uniform(self.workspace_bounds['ymin'], self.workspace_bounds['ymax']),
        random.uniform(self.workspace_bounds['xmin'], self.workspace_bounds['xmax']),
        random.uniform(self.workspace_bounds['ymin'], self.workspace_bounds['ymax'])
    ]
    print(f"\nRandom Start: ({coords[0]:.2f}, {coords[1]:.2f})")
    print(f"Random Goal: ({coords[2]:.2f}, {coords[3]:.2f})")
    return coords

def display_menu(self) -> Optional[str]:
    """Display user menu and get selection"""
    print("\n--- Motion Planner UI ---")
    print("Choose an option:")
    print(" 1. Manually enter start/goal")
    print(" 2. Use randomly generated start/goal")
    print(" q. Quit")
    return input("Enter your choice (1/2/q): ").strip().lower()

def publish_coordinates(self, coords: List[float]) -> None:
    """Publish coordinates to /start_goal topic"""
    msg = Float64MultiArray(data=coords)
    self.trajectory_received = False
    self.start_goal_pub.publish(msg)
    print("\nSent start and goal to /start_goal. Waiting for trajectory...")

def wait_for_trajectory(self) -> None:
    """Wait until trajectory is received or node is shutdown"""
    rate = rospy.Rate(10) # 10Hz
    while not rospy.is_shutdown() and not self.trajectory_received:
        rate.sleep()

def run(self) -> None:
    """Main execution loop"""
    while not rospy.is_shutdown():
        choice = self.display_menu()

        if choice == 'q':
            break

        if choice == '1':
            coords = self.get_user_coordinates()
        elif choice == '2':
            coords = self.generate_random_coordinates()
        else:
            print("Invalid choice. Please try again.")
            continue

        self.publish_coordinates(coords)
        self.wait_for_trajectory()

if __name__ == '__main__':
    try:
        planner = MotionPlanner()
        planner.run()
    except rospy.ROSInterruptException:

```

In my motion planner, similar to the `rrt_node` file, I initialized a `motion_planner` node and created a publisher sending start/goal coordinates and a subscriber to receive the trajectories ultimately, setting up the communication channels. Then there is workspace configurations where I defined the acceptable boundaries for coordinate generation.

The next function deals with trajectory handling where it will process incoming data and prints each point out within the configuration bounds. The first function deals with receiving user inputs for the start and end goal positions and it will also handle any errors that the user inputs that is not valid. The next function deals with the randomization option where the user could choose to randomize their start and end goal points instead of manually entering them. To tie this all up, there will be a menu interface for the user to select their option with '1' being user generated coordinate points or '2' being randomly generated points or 'q' to quit the program.

After receiving the coordinates, the program will take these coordinates into a ROS message and send them to the path planner where the ROS will have wait until the path data arrives. In the main execution loop, it will continuously display the menu and process user inputs until the program is shutdown.